

포트폴리오

최우석

요약

Unity 기반 게임 클라이언트 개발자로, AOS/iOS 모바일 게임, Steam 네트워크 게임, 아케이드 하드웨어 연동, 출시 프로젝트 글로벌 운영 대응을 경험했습니다.

기술 스펙

Language	C#, Unity C#, C++, JavaScript
Client	Unity, URP, Addressables, Shader/HLSL, UI/UX
Mobile	AOS/iOS 출시, IAP, Ads(AdMob, IronSource), 보안/암호화, Firebase
Network	Photon Fusion 2, Photon PUN, Host Migration, REST API
Build / I/O	Steam/Android/iOS/Windows 빌드, Unity Editor Tool, SerialPort, QR/RF 하드웨어 연동

프로젝트 구성

프로젝트	역할	핵심 내용
내맘대로마법사	개발·기획·팀장	AOS/iOS 모바일 게임. 아웃게임, 서버/시트 연동, 스킬/이펙트 구조 담당.
Dragon Arena	메인 개발	Steam 5v5 MOBA. Photon Fusion 전투, 드래곤 오브젝트, 스킬 구조 담당.
RaidOne	메인 개발	Steam 1v5 보스 레이드. Host Migration, 네트워크 전투, 보스/영웅 스킬 구조 담당.
메탈카드봇 듀얼아레나	메인 개발	아케이드 하드웨어 연동. SerialPort, QR/RF, 카드 배출, 전투/빌드 구조 담당.
데몬스워드 키우기 글로벌	단독 유지보수	출시 프로젝트 글로벌 대응. SDK/광고/IAP/보안, 서버 이벤트/빌드 안정화

대표 역량

- 라이브 운영: 상점, 광고, IAP, 서버/시트 데이터 연동, Remote Config
- 전투 구현: 스킬 인터페이스, 데미지 계산, 히트 판정, Shader
- 네트워크: Photon Fusion 2, RPC, Host Migration, Network Object Pooling
- 외부 연동: Firebase, Steamworks, SerialPort, QR/RF, 플랫폼별 빌드 대응

내맘대로마법사

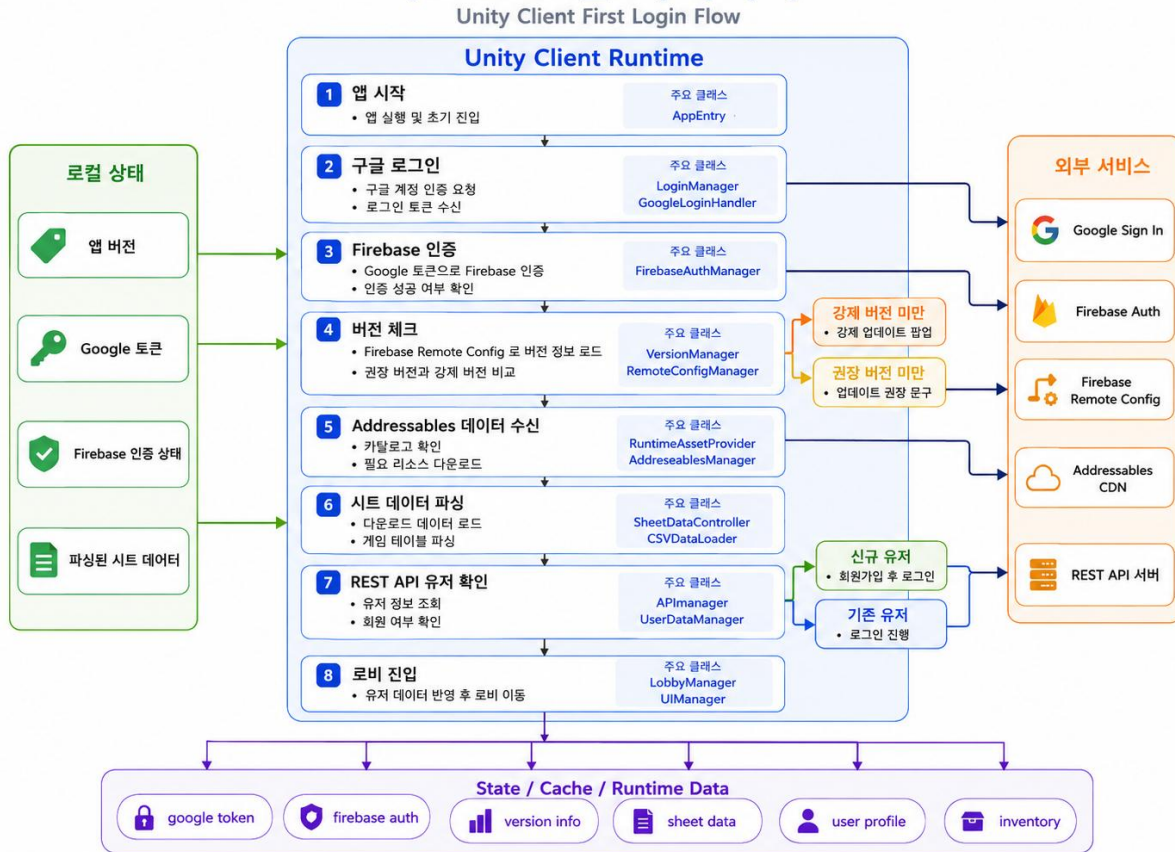
사용 기술	Unity URP, C#, AOS, Firebase, Addressables, REST API, Google Sheet	개발 정보	팀 프로젝트 / 개발·기획·팀장
역할	개발·기획·팀장. 팀 구성: 기획 1, 서버 1, 개발 3, 디자이너 1. 기여도 약 70%. 아웃게임 대부분과 인게임 스킬/VFX 구조 담당.		
내용	프로젝트 소개 Unity URP 모바일 게임입니다. 다양한 스킬과 속성 시너지를 조합해, 유저가 전투 중 성장 방향을 선택하는 구조의 게임입니다.		
	관련 링크 플레이 영상: https://youtu.be/0XKcMJXp39Y		
	주요 작업 • 상점, 장비, 스킬, 가차, 광고, IAP, 닉네임, 설정 등 아웃게임 기능 구현 • Firebase Remote Config 기반 버전 관리와 업데이트 안내 흐름 구현 • Google Sheet/서버 데이터 연동과 유저 데이터 반영 흐름 구현 • Addressables 기반 콘텐츠/프리팍 로드 구조 구현 • 스킬 인터페이스, 스킬 매니저, 데미지 계산, VFX/Shader 제작		
	성과 • 아웃게임과 인게임 스킬 흐름을 계층별로 분리해 콘텐츠 추가와 밸런스 조정 대응		

	범위를 줄였습니다. • 데이터, 콘텐츠, 전투 로직의 책임을 나눠 모바일 게임 구조에 맞게 정리했습니다. • 전투 중 몬스터와 이펙트가 많아질 때 발생하는 프레임 드랍을 풀링과 생성 흐름 정리로 완화했습니다.
--	---

아키텍처

외부 데이터, 콘텐츠, 아웃게임, 인게임 전투 계층으로 나뉘었습니다. 서버/시트 데이터는 부트 단계에서 정리되고, 로비는 유저 보유 데이터와 상점/가차/IAP 를 처리하며, 인게임은 스테이지 로드·스폰·스킬·데미지 계산 흐름으로 분리됩니다.

최초 로그인 아키텍처



개발 내용

개발 내용	기능 구현 - 초기 로그인, 버전 체크, 데이터 수신 흐름 구현 - 상점, 장비, 스킬, 가차, 광고, IAP 결과가 유저 데이터에 반영되는 흐름 구현 - 스킬 생성, 범위 판정, 데미지 계산, 속성/시너지 효과 적용 구조 구현 - 스킬 이펙트, 룬/정령 관련 UI, 로딩/가차 연출 제작
-------	---

	최적화 • 스테이지 진행 중 반복 생성되는 스킬/몬스터/이펙트 흐름을 풀링 구조로 정리 • 시트 데이터 파싱과 스테이지 데이터 사용 흐름을 분리해 로딩 이후 런타임 부담을 줄임 • 스킬 데미지 계산을 공통 처리로 묶어 유지보수 비용을 낮춤
	리팩토링 • 콘텐츠 로드 호출부와 실제 사용부를 분리해 교체 가능한 구조로 정리 • 스킬별 데미지 적용을 공통 계산 흐름으로 정리 • 스킬 추가 시 매니저, 인터페이스, 판정 처리 수정 범위를 줄이는 방향으로 구조화

내맘대로마법사 - 기능 이미지 / 코드 정리

01. Skill / VFX Runtime

인게임 스킬 연출, 데미지 계산, 독립 실행형 스킬 타겟 탐색 구조를 한 묶음으로 정리했습니다.

	<pre>public interface ISkillDamage { void OnSkillDamage(SkillInterface skill, IDamageable damageable, float damageMultiplier = 1f, Vector3 hitPosition = null); } public class SkillDamage : ISkillDamage { public void OnSkillDamage(SkillInterface skill, IDamageable damageable, float damageMultiplier = 1f, Vector3 hitPosition = null) { if (skill == null damageable == null skill.player == null) return; BuffManager buffManager = skill.player.BuffManager; if (buffManager == null) return; Transform targetTransform = damageable.GetTransform(); if (hitPosition != null) skill.HitAction.Invoke(targetTransform); float elementalDamage = GetElementalDamage(skill, buffManager); float elementalBonus = buffManager.GetValue(BuffType.ElementalBonus) > 0f ? buffManager.GetValue(BuffType.ElementalBonus) : 0f; float bonusDamage = skill.CurDamage; float bonusDamage = Mathf.Max(1f, buffManager.GetValue(BuffType.DamageBuff)); float elementalBonus = Mathf.Max(1f, elementalDamage); float finalDamage = bonusDamage * elementalBonus * elementalBonus * Mathf.Max(1f, damageMultiplier); damageable.TakeDamage((int)finalDamage, skill.ElementType, elementalBonus); } private static float GetElementalDamage(SkillInterface skill, BuffManager buffManager) { bool synergyActive = skillManager != null && skillManager.SynergySkill != null && skillManager.IsMatchingSynergy(skill.ElementType) && skillManager.SynergySkill.IsOnSkill(); if (!synergyActive) return 1f; switch (skill.ElementType) { case ElementType.Fire: return buffManager.GetValue(BuffType.FireDamageBuff); case ElementType.Water: return buffManager.GetValue(BuffType.WaterDamageBuff); case ElementType.Electric: return buffManager.GetValue(BuffType.ElectricDamageBuff); case ElementType.Land: return buffManager.GetValue(BuffType.LandDamageBuff); default: return 1f; } } }</pre>
인게임 스킬/VFX 연출 화면	스킬 데미지/속성 시너지 공통 계산 코드
<pre>public interface ISingletonSkillBehavior : ISkillBehavior { bool RequiresSkillObjectPool { get; } } public struct RuntimeSkillHitResult { public IDamageable Damageable; public Collider2D Collider; public Vector3 HitPosition; public RuntimeSkillHitResult(IDamageable damageable, Collider2D collider, Vector3 hitPosition) { Damageable = damageable; Collider = collider; HitPosition = hitPosition; } }</pre>	<pre>public static IDamageable FindFirstDamageable(Vector3 center, float radius, IDamageable ignored = null) { return TryFindFirstDamageable(center, radius, ignored, out RuntimeSkillHitResult result) ? result.Damageable : null; } public static bool TryFindFirstDamageable(Vector3 center, float radius, IDamageable ignored, out RuntimeSkillHitResult result) { result = default; int hitCount = Physics2D.OverlapCircle(center, Mathf.Max(0.85f, radius), QueryContactFilter, QueryBuffer); float closestSqr = float.MaxValue; for (int i = 0; i < hitCount; i++) { Collider2D hitCollider = QueryBuffer[i]; if (TryReactWith(hitCollider)) continue; IDamageable damageable = GetDamageable(hitCollider); if (damageable == null damageable == ignored) continue; Vector3 hitPosition = GetClosestPoint(hitCollider, center); float sqr = (hitPosition - center).sqrMagnitude; if (sqr < closestSqr) continue; closestSqr = sqr; result = new RuntimeSkillHitResult(damageable, hitCollider, hitPosition); } return result.Damageable != null; }</pre>
독립형 런타임 스킬 인터페이스/히트 결과 구조	타겟 탐색 및 피격 위치 계산 코드

02. Loading / Addressables Runtime Assets

로딩 단계에서 필요한 런타임 에셋을 Addressables 기반으로 요청하고, 스킬 프리팹은 로드 후 풀로 준비하는 흐름입니다.

	<pre>public static void LoadAssetAsync<T>(string key, Action<RuntimeAssetResult<T>> completed) where T : UnityEngine.Object { AsyncOperationHandle<T> handle = Addressables.LoadAssetAsync<T>(key); handle.Completed += completedHandle => { bool succeeded = completedHandle.Status == AsyncOperationStatus.Succeeded && completedHandle.Result != null; if (succeeded) { loadedAddressablesHandles.Add(completedHandle); completed?.Invoke(new RuntimeAssetResult<T>(key, completedHandle.Result, true, RuntimeAssetBackend.Addressables, completedHandle)); return; } if (completedHandle.IsInvalid()) { Addressables.Release(completedHandle); } if (FallbackToResourcesOnAddressablesFailure) { LoadResourceAssetAsync(key, completed); return; } completed?.Invoke(new RuntimeAssetResult<T>(key, null, false, RuntimeAssetBackend.Addressables, completedHandle)); } }</pre>
로딩 화면 / 리소스 준비 상태 표시	Addressables.LoadAssetAsync 런타임 에셋 로드
<pre>void CreateSkill(int _num, string _skillname, int _count) { if (skillkeyvalue[_num].SkillBehavior is StandaloneSkillBehavior standaloneSkill && standaloneSkill.RequiresSkillObjectPool) { return; } Action<HitCollider> initializeHitCollider = (HitCollider hitCollider) => { hitCollider.SkillSetting(skillkeyvalue[_num]); }; RuntimeAssetProvider.LoadAssetAsync<GameObject>(_skillname + "_Upgrade", result => { if (result.Succeeded) { GameObject prefab = result.Result; AddressableHandler.Instance.ReplaceShadersAll(prefab); if (!upgradeskillObjectPools.ContainsKey(_num)) { upgradeSkillObjectPools[_num] = new ObjectPool<HitCollider>{ prefab.GetComponent<HitCollider>(), _count, this.transform, initializeHitCollider }; } else { Debug.LogError(\$"Failed to load {_skillname}_upgrade prefab."); } } }); RuntimeAssetProvider.LoadAssetAsync<GameObject>(_skillname + "_Default", result => { if (result.Succeeded) { GameObject prefab = result.Result; AddressableHandler.Instance.ReplaceShadersAll(prefab); if (!defaultSkillObjectPools.ContainsKey(_num)) { defaultSkillObjectPools[_num] = new ObjectPool<HitCollider>{ prefab.GetComponent<HitCollider>(), _count, this.transform, initializeHitCollider }; } else { Debug.LogError(\$"Failed to load {_skillname}_default prefab."); } } }); }</pre>	로딩 처리 흐름 - Addressables 에셋 요청 - 로드 완료 핸들 보관 - 스킬 프리팹 로드 후 풀 생성 - 필요한 리소스 준비 후 다음 씬 진입
스킬 프리팹 Default/Upgrade 로드 및 풀 생성	로딩 / 에셋 준비 흐름

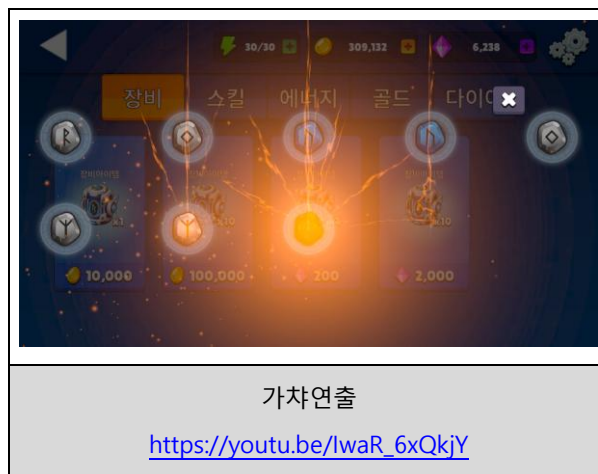
03. Loading / Sheet Data Pipeline

시트 메타 데이터 확인, 버전 비교, 필요한 시트 다운로드와 암호화 저장 흐름입니다.

<pre> www.result = SheetRequest.Result.Success; float metaStartTime = Time.realtimeSinceStartup; yield return www.SendWebRequest(); float metaSeconds = Time.realtimeSinceStartup - metaStartTime; if (www.result != UnityWebRequest.Result.Success) { Debug.LogError("meta download failed: " + www.error); isLoadingDone = false; yield break; } //mainSheetData(mainDownloadHandler.text); LogSheetProfile("meta network (metaSeconds:0.000)s, sheets (mainSheetData.Count)"); SheetVersionList localVersions = LoadLocalVersions(); bool anyUpdated = false; IList<MainSheetData> sheetsToUpdate = new List<MainSheetData>(); foreach (var sheet in mainSheetData) { int localVersion = localVersions.GetVersion(sheet.sheet_name); if (localVersion < sheet.versionnum GameManager.Instance.IsDev) sheetsToUpdate.Add(sheet); } int nextIndex = 0; int activeCount = 0; int completedCount = 0; int successCount = 0; if (downloadLimit < Mathf.Max(1, Mathf.Min(sheetsToUpdate.Count))); while (completedCount < sheetsToUpdate.Count) { while (activeCount < downloadLimit && nextIndex < sheetsToUpdate.Count) { MainSheetData sheet = sheetsToUpdate[nextIndex++]; activeCount++; StartCoroutine(DownloadAndSaveEncrypted(sheet, success => { activeCount--; completedCount++; if (!success) return; localVersions.UpdateVersion(sheet.sheet_name, sheet.versionnum); anyUpdated = true; successCount++; })); } yield return null; } if (anyUpdated) GameManager.Instance.SaveSheetData(); </pre>	<pre> IEnumerator DownloadAndSaveEncrypted(MainSheetData meta, Action<bool> completed = null) { UnityWebRequest req = UnityWebRequest.Get(fixedURL + meta.gid); req.timeout = SheetRequestTimeoutSeconds; float startTime = Time.realtimeSinceStartup; yield return req.SendWebRequest(); float networkSeconds = Time.realtimeSinceStartup - startTime; if (req.result != UnityWebRequest.Result.Success) { Debug.LogError(meta.sheet_name + " download failed: " + req.error); completed?.Invoke(false); yield break; } string tsv = req.downloadHandler.text; float encryptStartTime = Time.realtimeSinceStartup; string filePath = GetEncryptedFilePath(meta.sheet_name); Task encryptTask = Task.Run(() => { byte[] encrypted = AesCryptoUtil.EncryptStringToBytes(tsv); File.WriteAllBytes(filePath, encrypted); }); while (!encryptTask.IsCompleted) yield return null; float encryptSeconds = Time.realtimeSinceStartup - encryptStartTime; if (encryptTask.IsFaulted) { Debug.LogError("Sheet encrypt/write failed: " + meta.sheet_name + " " + encryptTask); completed?.Invoke(false); yield break; } tsvCache[meta.sheet_name] = tsv; LogSheetProfile(\$"sheet (meta.sheet_name) network (networkSeconds:0.000)s, encrypt/wri completed?.Invoke(true); </pre>
시트 메타/버전 비교 및 병렬 다운로드 큐	시트 다운로드 후 AES 암호화 저장 처리
시트 데이터 처리 - 메타 시트에서 버전 확인 - 로컬 버전보다 높은 시트만 갱신 - 다운로드 후 AES 암호화 저장 - 로딩 단계에서 데이터 준비 완료 대기	로딩 문구 예시 - Loading data... - Loading resources... - Loading complete
데이터 처리 요약	로딩 단계 표시

03. 아이템 가차 이펙트 및 연출제작

가차에 필요한 연출과 이펙트를 제작하였습니다..



Dragon Arena

사용 기술	Unity, C#, Steam PC, Photon Fusion 2, Steamworks.NET, Build Automation	개발 정보	팀 프로젝트 / 메인 개발
역할	메인 개발. 팀 구성: 기획 1, 디자이너 1, 개발 2. 개발 기여도 약 60%. RaidOne 기반 전투 구조를 5v5 MOBA 규칙으로 확장.		
내용	프로젝트 소개 Steam PC 5v5 MOBA 프로젝트입니다. 팀 전투, 드래곤 오브젝트, 몬스터 스폰, 스킬 모듈과 네트워크 전투 흐름을 담당했습니다.		
	관련 링크 Steam: https://store.steampowered.com/app/4371940/Dragon_Arena/ 트레일러 영상: https://youtu.be/okkGrxD1XE0		
	주요 작업 • 5v5 규칙에 맞는 팀 전투, 드래곤 오브젝트, 승패 흐름 구현 • 몬스터 스폰, 스폰 포인트, 미니맵 아이콘, 오브젝트 상태 동기화 • Projectile, Range, Buff 계열 스킬 모듈 구현 • Steam 로그인/IAP 와 서버 유저 데이터 연동 흐름 구현		
	성과		

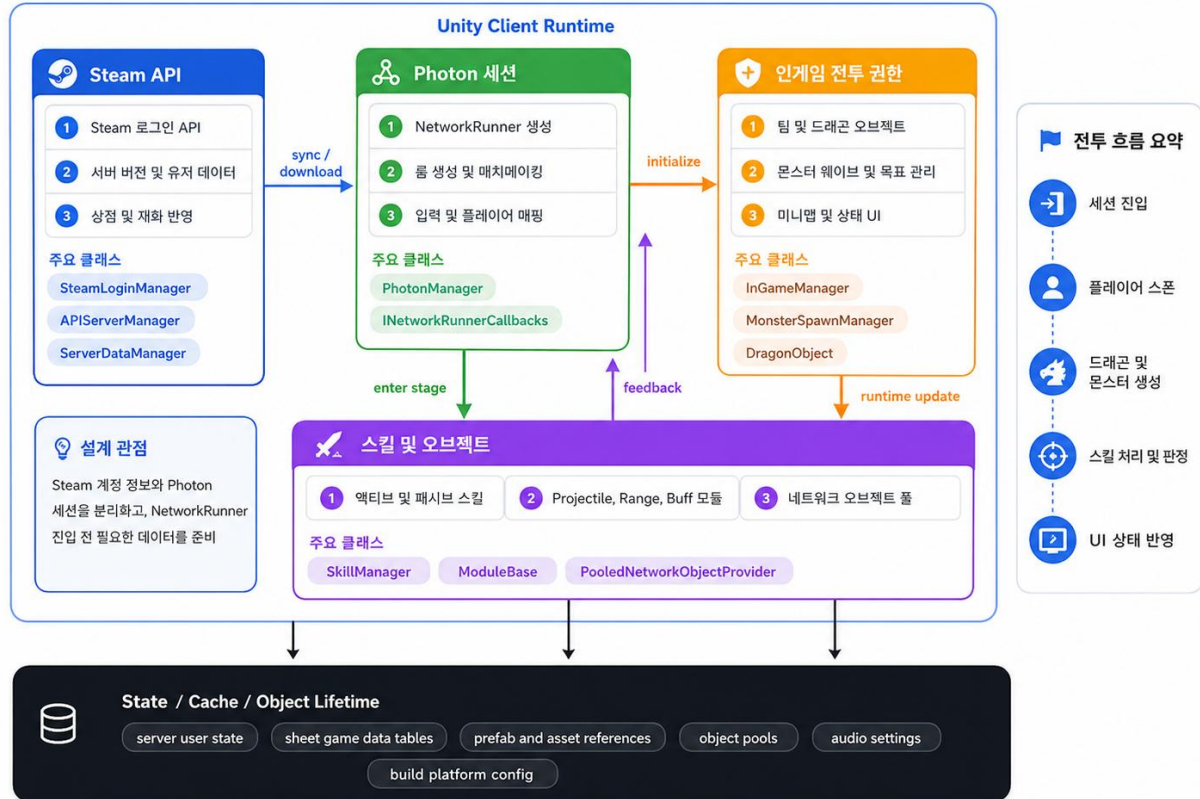
	• RaidOne 기반 전투 구조를 팀 기반 목표 전투 구조로 확장했습니다. • Steam 출시 프로젝트에서 실시간 네트워크 전투와 오브젝트 UI 흐름을 담당했습니다.
--	---

아키텍처

Steam/API 계층, Photon 세션 계층, 인게임 권한/동기화 계층, 스킬/오브젝트 계층으로 설계했습니다. Steam 로그인과 서버 유저 데이터는 로비에서 처리하고, 인게임에서는 NetworkRunner 권한 기준으로 플레이어·드래곤·몬스터·스킬 상태를 동기화합니다.

Dragon Arena 전투 아키텍처

System architecture / combat runtime flow / responsibility split



아키텍처 구조도

개발 내용

개발 내용	기능 구현 - 팀 전투 방식 구현(기본 전투, 패링) - 몬스터 스폰, 미니맵, 상태 UI 등 전투 정보 표시 흐름 구현 - 액티브/패시브 스킬과 모듈 기반 효과 구조 구현
-------	---

최적화

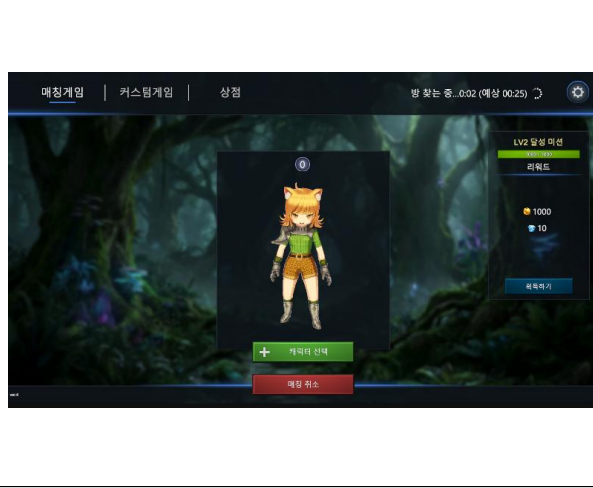
- 전투 중 생성되는 네트워크 오브젝트의 생성/반환 흐름을 분리
- 스킬 모듈과 이펙트 생성 흐름을 분리해 스킬 추가 비용을 낮춤
- AI 반응과 목표 오브젝트 처리 우선순위를 정리

리팩토링

- RaidOne 의 공통 전투 구조와 Dragon Arena 전용 MOBA 규칙을 분리
- 스킬 계산과 실제 네트워크 오브젝트 생성 책임을 나눠 유지보수성을 높임

매칭, 대기방, 전투, 미니맵 Fog of War, 네트워크 오브젝트 풀링 중심으로 정리했습니다.

점수 범위, 번들 버전, 인원 조건을 기준으로 매칭 세션을 찾고 StartGame 인자로 방 정보를 구성하는 흐름입니다.

	<pre>private SessionInfo? FindBestMatchmakingSession(int myScore) { int min = myScore - MATCH_ELO_RANGE; int max = myScore + MATCH_ELO_RANGE; int myBundle = ServerDataManager.Instance.BundleVersion(); var candidates = sessionList.Where(s => { if (!s.IsOpen) return false; if (!s.IsVisible) return false; if (!s.Properties.TryGetValue("Mode", out var mode)) return false; if (!"TM".Equals((string)mode)) return false; if (!s.Properties.TryGetValue("ScoreMin", out var pmin)) return false; if (!s.Properties.TryGetValue("ScoreMax", out var pmax)) return false; if (!s.Properties.TryGetValue("BundleVersion", out var bver)) return false; int roomBundle = (int)bver; if (roomBundle != myBundle) return false; int roomMin = (int)pmin; int roomMax = (int)pmax; bool allowed = (myScore >= roomMin && myScore <= roomMax); int curPlayers = s.PlayerCount; bool hasSlot = curPlayers < MATCH_SIZE; return allowed && hasSlot; }).ToList(); if (candidates.Count == 0) return null; return candidates.OrderByDescending(c => c.PlayerCount).First(); }</pre>
매칭 화면	매칭 세션 필터링 코드
	<pre>void InitWaiting() { SceneManager.LoadScene(SceneManager.GetActiveScene().BuildIndex + 1); } var member = GameObject.Find<RoomData>(xindex); var args = new StartGameArgs() { GameId = null, SessionName = SessionName, Done = Member, SceneManager = _runner.GetComponent<UISceneManager>?.Fallback(), PlayerCount = MATCH_SIZE, IsOpen = true, IsVisible = isVisible, SessionProperties = args, ConnectionToken = connection, WaitingRoomName = waitingRoomName, MigrationToken = null }; isVisible = isVisible; serverWaitCount = MATCH_SIZE; var result = await _runner.StartGame(args); if (result.Ok) { isClosed = false; await AsyncVoidIf(Lifetime 종료 [result.ShutdownReason]); Init().Forget(); if (result.ShutdownReason == ShutdownReason.ConnectionFailed) { SceneManager.Instance.CloseDialogPanel(); SceneManager.Instance.OpenDialogPanel("ConnectionFailed"); } return; } SceneManager.Instance.CloseDialogPanel(); await AsyncVoidIf(PushMessage) { Shut-Down 중입니다 }; if (forWaitingRoom && !isWaitingRoom) { await AsyncVoidUntil(() => waitingRoomManager.Instance && waitingRoomManager.Instance.waitingRoomPanel); SceneManager.Instance.LoadSceneAsync("waitingRoom").Forget(); await AsyncVoidIf(PushMessage) { waitingRoom 전 로딩 완료되었습니다 }; } if (_runner.IsServer) { await AsyncVoidIf(PushMessage) { Starting periodic snapshot (host only)! }; StartCoroutine(SnapshotPushMessage()); StartCoroutine(Intern, 10, / second); StartCoroutine(Cancel, 10, / second); } waitingRoomPanel = waitingRoomManager.Instance.waitingRoomPanel; waitingRoomPanel.RefreshInfo(isVisible); } else { if (_runner.IsServer) { await AsyncVoidIf(PushMessage) { Host active, waiting for players to reach 4...! }; StartCoroutine(SnapshotPushMessage()); } }</pre>
대기방 / 팀 구성 화면	StartGameArgs 구성 및 세션 시작 코드

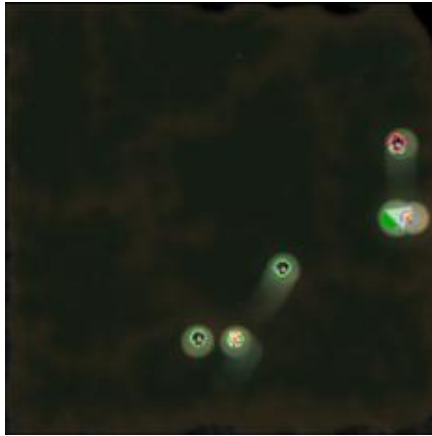
02. Battle / Network Runtime

전투 화면, AI 입력 생성, 데미지 처리, NetworkObject 풀링 흐름입니다.

	<pre>public void OnNetworkObjectAcquireResult(AcquirePrefabInstance(NetworkRunner runner, INetworkTranAcquireContext context, INetworkObject instance) { instance = null; if (DelayIfSceneManagerIsBusy && runner.SceneManager.IsBusy) { return NetworkObjectAcquireResult.Retry; } NetworkObject prefab; if (runner.TryGetPrefab(context.PrefabId, out prefab, context.IsSynchronous)) { Debug.Log(\$"[Pool] Failed to load prefab: {context.PrefabId}"); return NetworkObjectAcquireResult.Failed; } if (prefab) { return NetworkObjectAcquireResult.Retry; } if (_pool.TryGetPrefab(context.PrefabId, out var queue) && queue.Count > 0) { instance = queue.Dequeue(); instance.gameObject.SetActive(true); } else { instance = InstantiatePrefab(runner, prefab); } if (context.IsSynchronous) { runner.NetworkObjectAcquireResult.OnSuccess(instance.gameObject); } else { runner.NetworkObjectAcquireResult.OnSuccess(instance.gameObject); } return NetworkObjectAcquireResult.Success; }</pre>
5v5 전투 화면	NetworkObject Acquire / Pooling 코드
<pre>private bool AI_BuildInput(out NetworkingInputData input) { input = default; if (!CanAIControl NetIsDeath !IsAlive) return false; // 1. 매 프레임 타겟을 먼저 찾음 (뒤에 있는 몬스터 감지) AI_TryAcquirePriorityTarget(); AI_UpdateChaseTarget(); // 2. 이동 방향 계산 Vector3 _dirWorld = AI_GetMovedirectionWorld(); if (_dirWorld.sqrMagnitude < 0.0001f) { input.moveDir = Vector2.zero; input.isRun = false; // 움직임을 매 라임 주면 if (AI_IsChasing && AI_HasChaseTarget()) { Vector3 _face = AI_GetChaseTargetPosition() - transform.position; _face.y = 0f; input.lookDelta = AI_ComputeLookDelta(_face.normalized); } AI_ServerUpdateCombat(ref input); return true; } _dirWorld.y = 0f; _dirWorld.Normalize(); Vector2 _md; _md.x = Vector3.Dot(transform.right, _dirWorld); _md.y = Vector3.Dot(transform.forward, _dirWorld); input.moveDir = Vector2.ClampMagnitude(_md, 1f); input.isRun = true; // 달음을 쫓거나 이동할 때 항상 달음 설정 input.lookDelta = AI_ComputeLookDelta(_dirWorld); AI_ServerUpdateCombat(ref input); return true; }</pre>	<pre>public void DamageOwner(float damage, MonoBehaviour attacker, bool isCritical, bool applyHitSkill = true, bool cameraShake = true) { if (IsInvincible() !IsAlive Evald()) { return; } NPC_CancelReturn(); #region Attacker 기록 if (Object.HasStateAuthority) { float now = (float)Runner.SimulationTime; if (attacker is Player _attackerPlayer) { if (_attackerPlayer.PlayerInfoData != null && PlayerInfoData != null && _attackerPlayer.PlayerInfoData.Team != PlayerInfoData.Team) { HITTimeByPlayers[_attackerPlayer.Object.Id] = _now; lastHitByPlayerId = _attackerPlayer.Object.Id; lastHitTime = _now; hasLastHitByPlayer = true; } if (CanAIControl) { allLastAttackerId = _attackerPlayer.Object.Id; allLastAttackerTime = _now; } } else if (attacker is Monster _attackerMonster) { if (CanAIControl && _attackerMonster.Object != null) { allLastAttackerId = _attackerMonster.Object.Id; allLastAttackerTime = _now; } } } }</pre>
AI 입력 생성 및 추적 방향 계산	공격자 기록 / 피격 처리 코드

03. Minimap Fog of War

플레이어 주변만 미니맵에서 밝히는 Fog of War 처리와 방어/패링 이후 최종 데미지 반영 흐름입니다.



미니맵 Fog of War 화면

```
void StepReveal()
{
    CleanupRevealers();

    Camera _cam = Manager.MinimapCamera;
    Transform _camTr = _cam.transform;

    float _halfW = _cam.orthographicSize;
    float _aspect = GetAspect(_cam);
    float _halfH = _halfW * _aspect;

    Vector3 _camPos = _camTr.position;
    Vector3 _camRight = new Vector3(_camTr.right.x, 0f, _camTr.right.z).normalized;
    Vector3 _camUp = new Vector3(_camTr.up.x, 0f, _camTr.up.z).normalized;

    if (_camRight.sqrMagnitude < 1e-6f) _camRight = Vector3.right;
    if (_camUp.sqrMagnitude < 1e-6f) _camUp = Vector3.forward;

    foreach (var kv in revealers)
    {
        Transform _tr = kv.Key;
        float _radiusWorld = kv.Value;

        Vector3 _pos = _tr.position;
        if (WorldToUV2(_pos, _camPos, _camRight, _camUp, _halfW, _halfH, out Vector2 _uv))
            continue;

        int _cx = Mathf.RoundToInt(_uv.x * (TEX_SIZE - 1));
        int _cy = Mathf.RoundToInt(_uv.y * (TEX_SIZE - 1));

        float _unitsPerPixelX = (_halfW * 2f) / TEX_SIZE;
        float _unitsPerPixelY = (_halfH * 2f) / TEX_SIZE;
        float _unitsPerPixel = Mathf.Max(0.0001f, (_unitsPerPixelX + _unitsPerPixelY) * 0.5f);

        float _rPix = _radiusWorld / _unitsPerPixel;
        float _softPix = SoftEdgeWorld / _unitsPerPixel;

        RevealCircle(_cx, _cy, _rPix, _softPix);
    }
}
```

Reveal 위치 계산 및 원형 밝힘 처리

```
void PushPixel()
{
    byte _baseA = (byte)Mathf.Clamp(Mathf.RoundToInt(fogAlpha * 255f), 0, 255);

    for (int i = 0; i < fogm.Length; i++)
    {
        byte _a = (byte)Mathf.Clamp(Mathf.RoundToInt(fogm[i] * _baseA), 0, 255);
        pixel[i] = new Color32(0, 0, 0, _a);
    }

    fogTex.SetPixels32(pixel);
    fogTex.Apply(false, false);
}

int GetAspect(Camera cam)
{
    if (cam == null)
        return 1f;

    if (cam.targetTexture != null)
    {
        float _w = Mathf.Max(1f, cam.targetTexture.width);
        float _h = Mathf.Max(1f, cam.targetTexture.height);
        return _w / _h;
    }

    return Mathf.Max(0.1f, cam.aspect);
}

bool WorldToUV2(
    Vector3 worldPos,
    Vector3 camPos,
    Vector3 camRight,
    Vector3 camUp,
    float halfW,
    float halfH,
    out Vector2 uv)
{
    Vector3 _d = worldPos - camPos;
    float _x = Vector3.Dot(_d, camRight);
    float _z = Vector3.Dot(_d, camUp);

    float _u = (_x / Mathf.Max(0.0001f, halfW) + 1f) * 0.5f;
    float _v = (_z / Mathf.Max(0.0001f, halfH) + 1f) * 0.5f;

    uv = new Vector2(_u, _v);

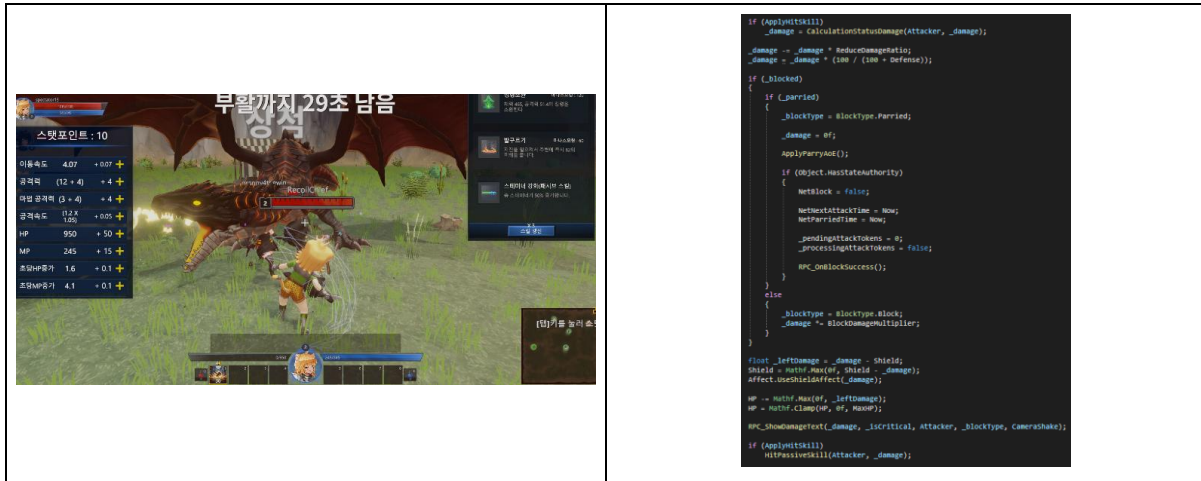
    if (_u < 0f || _u > 1f || _v < 0f || _v > 1f)
        return false;

    return true;
}
```

Fog 텍스처 픽셀 반영 / 월드 좌표 변환

03. 플레이어 전투

플레이어전투시 패링작업입니다.



RaidOne

사용 기술	Unity, C#, Steam PC, Photon Fusion 2, Steamworks.NET, Build Automation	개발 정보	팀 프로젝트 / 메인 개발
역할	메인 개발. 팀 구성: 기획 1, 디자이너 1, 개발 2. 개발 기여도 약 60%. 1v5 보스 레이드 전투와 네트워크 구조 담당.		
내용	프로젝트 소개 Steam PC 1v5 온라인 보스 레이드 프로젝트입니다. 비대칭 전투, 보스/플레이어 스킬, 몬스터 스폰, Host Migration 기반 세션 안정화 구조를 담당했습니다.		
	관련 링크 Steam: https://store.steampowered.com/app/3896480/Raid_One__1_vs_5_Online_Boss_Battle/		

주요 작업

- 1v5 보스 레이드 규칙과 보스/플레이어 역할 분리 구현
- Photon Fusion 기반 룸, 매치메이킹, Host Migration/재접속 대기 대응
- 몬스터 스폰, 소환체, 레이드 맵, 보스 스킬 흐름 구현
- Hero/Dragon 스킬 데이터 변환과 스킬셋 분리 구현

성과

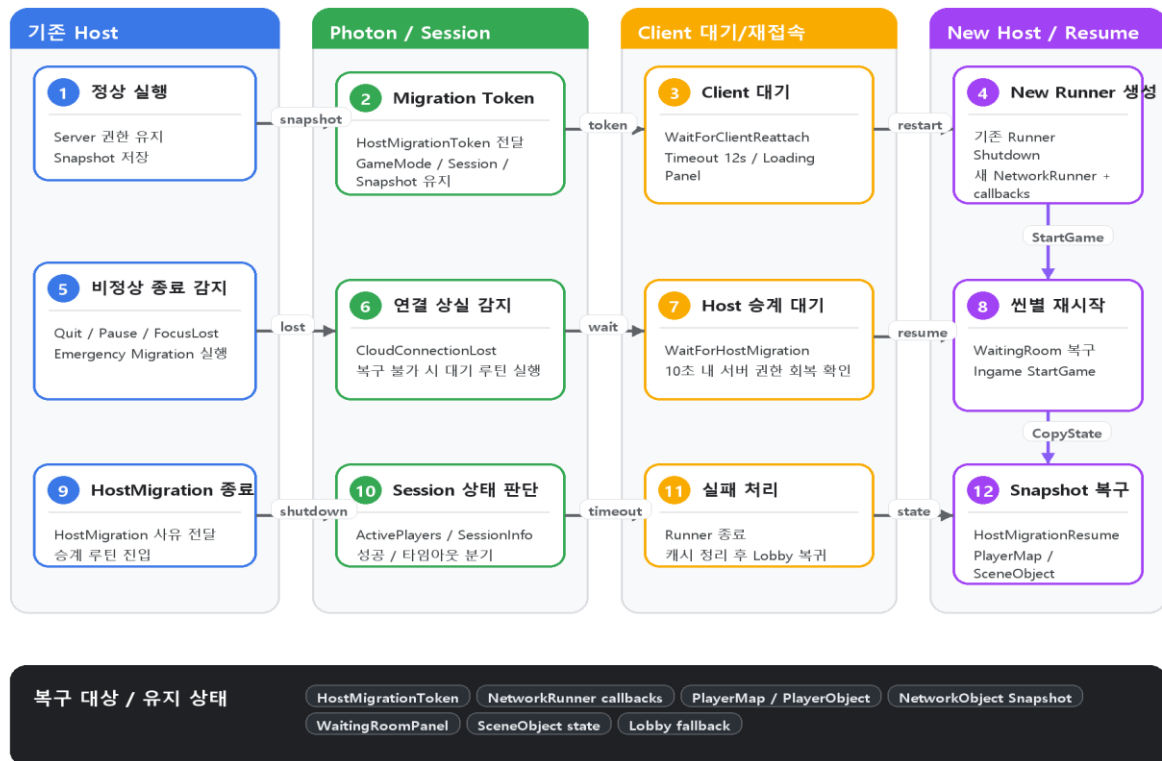
- 비대칭 전투에서 역할별 스킬·상태·스폰 흐름을 분리했습니다.
- HostMigrationToken, 재접속 대기, 스냅샷 복구 흐름으로 호스트 이탈 상황을 보완했습니다.

아키텍처

RaidOne 의 네트워크 구조는 Host Migration 상황을 기준으로 정리했습니다. 호스트가 종료되거나 연결을 잃으면 스냅샷을 저장하고, 클라이언트는 재접속 패널로 대기합니다. 이후 HostMigrationToken 으로 새 NetworkRunner 를 생성하고 WaitingRoom/Ingame 상태에 맞춰 StartGame 을 재시작한 뒤, HostMigrationResume 에서 PlayerMap, PlayerObject, SceneObject 상태를 복구합니다. 제한 시간 안에 복구되지 않으면 Runner 를 종료하고 로비로 복귀합니다.

RaidOne Host Migration / 재접속 구조

Photon Fusion 2 기반 세션 승계, 대기, 재접속, 스냅샷 복구 흐름



아키텍처 구조도

개발 내용

개발 내용	기능 구현 - 보스와 플레이어 스킬셋 분리 및 전투 상태 동기화 - 몬스터 스폰, 소환체, 레이드 맵 흐름 구현
-------	--

	• Host Migration, WaitingRoom 재구성, 클라이언트 재접속 대기 흐름 구현
	최적화 • NetworkObject 풀링으로 반복 생성되는 전투 오브젝트 비용 절감 • 주기 스냅샷과 긴급 스냅샷으로 호스트 이탈 시 복구 가능한 상태 유지 • 재접속 실패 시 캐시 정리와 로비 복귀를 분리해 세션 잔류 상태를 줄임
	리팩토링 • PhotonManager 에서 세션 생성, CloudConnectionLost, OnHostMigration, Resume 책임을 한 흐름으로 정리 • WaitingRoom 과 Ingame 복구 분기를 나눠 재접속 후 씬 상태가 꼬이지 않도록 구성

RaidOne - 기능 이미지 / 코드 정리

1 대 5 보스 레이드 프로젝트에서 재접속 대기, Host Migration, Snapshot Resume 복구 흐름을 중심으로 정리했습니다.

Steam: https://store.steampowered.com/app/3896480/Raid_One_1_vs_5_Online_Boss_Battle/

01. Reconnect / Client Reattach

CloudConnectionLost 발생 후 클라이언트가 Runner 와 Session 복구를 기다리고, 제한 시간 안에 복구되지 않으면 로비로 정리하는 흐름입니다.

	<pre>void OnCloudConnectionLost(NetServerRunner* runner, ShutdownReason reason, bool reconnecting) { Debug.Log("CloudConnectionLost: reason={reason}, reconnecting={reconnecting}"); _cloudReconnecting = reconnecting; // 재접속 대기 시작 if (reconnecting) MultiRequestMigrationReturnToLobby().Forget(); } void OnSessionSnapshotRestoreCompleted(NetServerRunner* runner, bool successful) { Debug.Log("SessionSnapshotRestoreCompleted: successful={successful}"); // 성공 시 재접속 대기 시작 _cloudReconnecting = reconnecting; // 로딩 화면 대기 후 새 세션 (리전)을 시작할, 준비된 상태 runner.Session.CloseLoadingPanel(); float waited = 0f; const float step = 0.1f; while (waited < maxWaitSeconds) { // 1. 세션 복구를 기다리고, 성공 시 재접속 대기 시작 if (_cloudReconnecting) { Debug.Log("CloudConnectionLost: 재접속 대기 시작"); runner.Session.CloseLoadingPanel(); return; } // 2. Runner가 실패 시 (reconnecting) 처리, 클라이언트가 다시 접속해제한 상태로 간주 if (_runner != null && !_runner.IsRunning) { // ActionPlayer가 복수 1. 리플레이 (리전)을 복원, SessionInfo를 복원하면 복구할 것으로 판단 bool playerBack = _runner.ActivePlayers != null && _runner.ActivePlayers.Count() > 0; bool sessionValid = _runner.SessionInfo != null && !_runner.SessionInfo.Invalid; if (playerBack && sessionValid) { Debug.Log("CloudConnectionLost: 세션 복구 완료"); runner.Session.CloseLoadingPanel(); return; } // 세션 복구 실패 시, 클라이언트가 다시 접속해제한 상태로 간주 waited += step; } // 리플레이를 다시 기다리고, 로딩 화면 Debug.Log("CloudConnectionLost: 리플레이를, 로딩 화면"); runner.Session.CloseLoadingPanel(); } try { if (_runner != null) { _runner.Shutdown(ShutdownReason.ShutdownReason.DM); } catch { } ClearPlayerCaches(); DestroyAllLocalPlayers(); MultiRequestReturnToLobby().Forget(); } }</pre>
재접속 대기 화면	CloudConnectionLost / Client Reattach 대기
	CloudConnectionLost 감지 reconnecting=false 이면 Host Migration 대기 Runner/Session 복구 감지 시 로딩 종료 Timeout 시 Runner 정리 후 Lobby 복귀
RaidOne 인게임 화면	재접속 처리 순서

기존 Runner 를 HostMigration 사유로 정리한 뒤 새 Runner 를 생성하고, WaitingRoom 과 Ingame 상태에 맞춰 StartGame 을 다시 구성하는 흐름입니다.

<pre> Debug.Log("OnHostMigration 호출됨"); // Host Shutdown foreach(var hostId in PushHostSnapshots()) { await Runner.Shutdown(hostId); } // Host Migration Debug.Log("HostMigration 기존 Runner Shutdown 완료"); await(jet.RunnerID == "newNetworkRunner") { transform = (parent * transform) }; var newRunner = runner.GetComponent<NetworkManager>(); newRunner.PrepareInput = true; newRunner.AddressList.Clear(); newRunner.GameObject.AddComponent<NetworkSceneManagerDefault>(); HostMigrationFailed(runner.ID); newRunner.Shutdown(); // HostMigration newRunner.ConnectionList.Clear(); newRunner.ConnectionList.Add(newConnectionList); connect = HostMigrationToken.GameMode == GameMode.Match; // WaitingRoom Room if (SceneLoader.Instance.CurrentSceneType == SceneType.WaitingRoom) { Debug.Log("HostMigration WaitingRoom은 현재까지 시작을 하지 않음"); SceneManager.LoadScene(WAITING_ROOM); } var props = new Dictionary<string, SerializableProperty> { { "CreatorID", SceneManager.Instance.DefinitionOf(HostMigration), }, { "NewPlayers", WAITING_ROOM } }; var result = await NewRunner.StartGame(New StartGameArgs { GameMode = HostMigrationToken.GameMode, Scene = SceneManager.DefaultSceneName, // HostMigrationToken.DefaultSceneName, // WaitRoom PlayerCount = WAITING_ROOM.Players, IsVisible = false, // HostMigrationToken.IsVisible IsOpen = true, SerializableProperties = props, SceneManagerInstance = HostMigrationToken, HostMigrationResume = HostMigrationToken.Resume, ConnectionToken = connectionToken }); if (!result.Ok) { Debug.LogError(\$"HostMigration WaitingRoom StartGame 실패 ({result.shutdownReason})"; SceneManager.Instance.OpenInfoPanel("HostMigrationFailed"); } else { // WaitingRoom D 종료/이동 if (SceneLoader.Instance.ActiveSceneName() != "WaitingRoom") SceneManager.Instance.LoadSceneAsync("WaitingRoom", LoadMode.Normal); if (newRunner.IsServer) </pre>	<pre> var ScenerRef = SceneLoader.Instance.CurrentSceneType == SceneType.Lobby ? SceneRef.FromIndex(1) : SceneLoader.Instance.CurrentSceneType == SceneType.WaitingRoom ? SceneRef.FromIndex(2) : null; var ingameResult = await newRunner.StartGame(new StartGameArgs { GameMode = HostMigrationToken.GameMode, IsOpen = false, IsVisible = false, Scene = ScenerRef, SceneManager = newRunner.GetComponent<NetworkSceneManagerDefault>(), HostMigrationToken = HostMigrationToken, HostMigrationResume = HostMigrationToken.Resume, ConnectionToken = connectionToken }); if (!ingameResult.Ok) { Debug.LogError(\$"HostMigration Ingame StartGame 실패: {ingameResult.shutdownReason}"); SceneManager.Instance.OpenInfoPanel("HostMigrationFailed"); } else { if (newRunner.IsServer) { Debug.Log(\$"HostMigration Registering periodic snapshot on new host"); InvokeRepeating(nameof(PushHostSnapshot), SNAPSHOT_INTERVAL_MS / 1000f, SNAPSHOT_INTERVAL_MS / 1000f); await newRunner.PushHostMigrationSnapshot(); } } } </pre>
HostMigration 진입 / 새 Runner 생성	Ingame StartGame / Snapshot 등록
기존 Snapshot 전송 중단 Runner Shutdown 후 새 Runner 구성 HostMigrationToken / Resume 콜백 전달 새 호스트에서 주기 Snapshot 재등록 Runner 재시작 흐름	WaitingRoom: 씬 재시작 후 방 상태 복구 Ingame: 현재 씬 유지 후 Snapshot Resume 실패 시 HostMigrationFailed 안내 상태별 복구 분기

Fusion Snapshot 에 남은 NetworkObject 를 다시 Spawn 하고, 플레이어 권한과 PlayerObject 매핑을 복구하는 구간입니다.

메탈카드봇 듀얼아레나

사용 기술	Unity, C#, Windows, SerialPort, RFID/QR, Spine, URP	개발 정보	팀 프로젝트 / 메인 개발
역할	메인 개발. 팀 구성: 기획 1, 디자이너 2, 개발 2. 개발 기여도 약 60%. 아케이드 하드웨어 연동과 전투 흐름 담당.		
내용	프로젝트 소개		

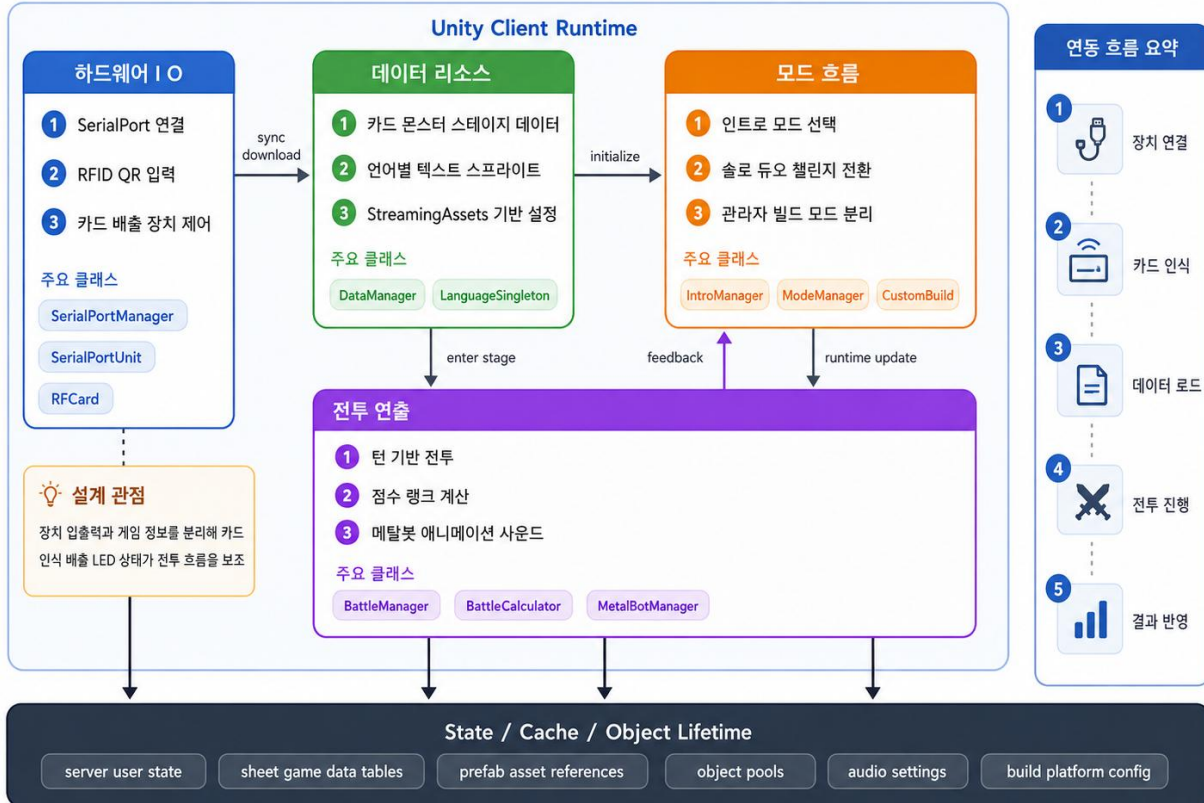
	메탈카드봇 애니메이션 IP 를 사용한 아케이드 하드웨어 연동 프로젝트입니다. 국가지원사업 프로젝트로 진행했으며 버튼클릭을 이용해 전투하는 게임입니다.
	관련 링크 영상/자료: https://drive.google.com/drive/folders/1TLnTK4uLL6UceQMtY69cZcjtKPVeOF7L
	주요 작업 • SerialPort 기반 카드 리더/배출 장치 연동 • RFID/QR 입력을 게임 카드 데이터와 연결 • 전투 매니저, 점수/랭크 계산, 메탈봇 애니메이션 제어 구현 • Computer/ARCADE 빌드 분리와 StreamingAssets 정리 자동화
	성과 • 하드웨어 입력과 Unity 전투 로직을 분리해 장치 예외 대응을 쉽게 만들었습니다. • 아케이드 설치 환경에 맞춰 빌드 모드와 운영 흐름을 분리했습니다. • 국가지원사업프로젝트를 성공리에 마감했습니다.

아키텍처

Serial/RFID/QR 계층은 장치 상태와 입력을 관리하고, Data/Language 계층은 카드와 전투 데이터를 제공합니다. Mode/Intro 계층에서 게임 흐름을 선택하고, Battle 계층은 전투 계산·연출·점수 처리를 담당합니다.

메탈카드봇 듀얼아레나 장치 연동 아키텍처

System architecture / device integration / runtime data flow



아키텍처 구조도

개발 내용

개발 내용	기능 구현 - 카드 인식, 카드 배출, QR/RF 입력 처리 흐름 구현 - 턴 기반 전투와 메탈봇 애니메이션/사운드 연동 - 게임 모드 선택, 관리자 흐름, 커스텀 빌드 메뉴 구현
-------	---

	최적화 • 장치 입력을 매니저 계층으로 묶어 전투 로직과 직접 결합되지 않게 정리 • 전투 점수 계산과 랭크 산출을 별도 계산 클래스로 분리 • 빌드 전 불필요한 데이터 정리와 빌드 타깃 분리 자동화
	리팩토링 • 하드웨어 이벤트, 카드 데이터, 전투 연출의 책임을 분리 • 모드별 씬 전환과 데이터 초기화 흐름을 정리

메탈카드봇 듀얼아레나 - 기능 이미지 / 코드 정리

아케이드 하드웨어와 Unity 클라이언트를 연동한 프로젝트로, 카드 인식, 시리얼 통신, 카드 배출, 전투 화면 흐름을 중심으로 정리했습니다.

Video / Drive: <https://drive.google.com/drive/folders/1TLnTK4uLL6UceQMtY69cZjtkPVeOF7L>

01. Cabinet / Card Recognition

RF 카드와 QR 카드 입력을 받아 게임 내 카드 데이터와 연결하고, 실제 기기 조작 흐름과 전투 진입을 맞춘 구간입니다.

	<pre>void DeviceInventoryCheck() { int iRet = 0; uint8_t* dlnvparamList = RFIDLib::rfidlib_reader_NDM_CreateInvenParamSpecList(); RFIDLib::rfidlib_alp_15015693.CreateInvenParam(dlnvparamList, (uint8_t*)0, (uint8_t*)0, (uint8_t*)0); iRet = RFIDLib::rfidlib_reader_NDM_TagInventory(header, RFIDLib::rfidlib_def_AJ_TYPR_MEM, 0, null, dlnvparamList); if (iRet != 0) { return; } uint8_t* tagdataReport = uint8Ptr.Zero; tagdataReport = RFIDLib::rfidlib_reader_NDM_GetTagDataReport(header, RFIDLib::rfidlib_def_RFID_SEEK_FIRST); //First while (tagdataReport != uint8Ptr.Zero) { TagInfo tag = new TagInfo(); uint8_t alp_id = 0; uint8_t tag_id = 0; uint8_t amt_id = 0; byte ddefid = 0; uint16_t uid = new byte[16]; string struid = ""; iRet = RFIDLib::rfidlib_alp_15015693.ISO15693_ParseTagDataReport(tagdataReport, ref alp_id, ref tag_id, ref amt_id, ref ddefid, uid); if (iRet == 0) { struid = BitConverter.ToString(uid, 0, 8).Replace("-", string.Empty); tag.uid = struid; tag.alp_id = alp_id; tag.tag_id = tag_id; tag.tag_amt = tag_id; tag.data = tag; if (cor != null) { stopCoroutine(cor); cor = null; } Debug.Log("카드 인식"); ConnectCard(struid); } tagdataReport = RFIDLib::rfidlib_reader_NDM_GetTagDataReport(header, RFIDLib::rfidlib_def_RFID_SEEK_NEXT); //next } }</pre>
QR 카드 인식 / 실기 화면	RFID 카드 UID 파싱 / 연결 코드
	
카드 배출 / 실기 화면	전투 화면

02. Serial Port / Device Setup

기기별 시리얼 리더와 카드 배출 체크 오브젝트를 초기화하고, XML 설정값으로 포트 정보를 분리한 구조입니다.

<pre>void Awake() { if (Instance == null) { cardTakeDeviceCheck[0] = true; cardTakeDeviceCheck[1] = true; Instance = this; DontDestroyOnLoad(gameObject); OpenSerialPort(); var go = new GameObject(); serialPortReader0 = go.AddComponent<SerialPortUnity>(); serialPortReader0.serialName = "SerialHeader1"; serialPortReader0.readType = true; serialPortReader0.OpenSerialPort(); cardoutQRChecker0 = go.AddComponent<cardoutQRChecker>(); cardoutQRChecker0.deviceName = 0; cardoutQRChecker0.serial = serialPortReader0; GameManager.Instance.unit1 = go.GetComponent<cardoutQRChecker>(); go.transform.SetParent(transform); go.name = serialPortReader0.serialName; go = new GameObject(); serialPortReader1 = go.AddComponent<SerialPortUnity>(); serialPortReader1.serialName = "SerialHeader2"; serialPortReader1.readType = true; serialPortReader1.OpenSerialPort(); cardoutQRChecker1 = go.AddComponent<cardoutQRChecker>(); GameManager.Instance.unit2 = go.GetComponent<cardoutQRChecker>(); cardoutQRChecker1.deviceName = 1; cardoutQRChecker1.serial = serialPortReader1; go.transform.SetParent(transform); go.name = serialPortReader1.serialName; } else { Destroy(gameObject); } }</pre>	<pre>void LoadXML() { XmlDocument xmlDoc = new XmlDocument(); xmlDoc.Load(Application.streamingAssetsPath + "/xml/SerialConfig.xml"); XmlNode root = xmlDoc.SelectSingleNode("SerialConfig"); // SerialPort XmlNode node = root.SelectSingleNode("SerialPort"); _PortName = node.Attributes["portName"].Value; _BaudRate = int.Parse(node.Attributes["baudRate"].Value); _Parity = (Parity)int.Parse(node.Attributes["parity"].Value); _DataBits = int.Parse(node.Attributes["dataBits"].Value); _StopBits = (StopBits)int.Parse(node.Attributes["stopBits"].Value); }</pre>
SerialPort Reader / Cardout Checker 초기화	SerialConfig.xml 포트 설정 로드
두 개의 카드 배출 장치 체크 Reader 오브젝트 런타임 생성 DontDestroyOnLoad 로 장치 상태 유지 포트명/보드레이트/패리티/XML 분리	Unity 클라이언트와 하드웨어 입력 분리 장치 교체 시 XML 설정만 수정 카드 입력/배출 이벤트를 게임 로직으로 전달
기기 초기화 기준	운영/유지보수 기준

03. Serial Packet / Read-Write

시리얼 패킷을 버퍼 단위로 읽고 종료 바이트를 기준으로 처리하며, 명령 전송은 고정 길이 버퍼로 구성했습니다.

<pre>void ReadBuffer() { if (!_SerialPort.IsOpen) return; try { while (true) { _ReadBuffer[_ReadBufferIndex] = (byte)_SerialPort.ReadByte(); if (_ReadBuffer[_ReadBufferIndex] == 0xEE) { LogReadBuffer(); ProcessReadBuffer(); ResetReadBuffer(); } else { if (++_ReadBufferIndex >= _BufferSize) ResetReadBuffer(); } } } catch (Exception e) { if (_ReadLastError != "ReadBuffer\n" + e.ToString()) { _ReadLastError = "ReadBuffer\n" + e.ToString(); Debug.Log(e); } } }</pre>	<pre>void WriteBuffer(SerialCode code, byte data1, byte data2, byte data3, float delay = 0) { if (!_SerialPort.IsOpen) return; _WriteBuffer[0] = (byte)code; _WriteBuffer[1] = data1; _WriteBuffer[2] = data2; _WriteBuffer[3] = data3; _WriteBuffer[4] = 0xEE; try { _SerialPort.Write(_WriteBuffer, 0, 5); } catch (TimeoutException e) { Debug.Log("WriteBuffer: " + e.ToString()); } catch (IOException e) { Debug.Log("WriteBuffer IOException\n" + e.ToString()); } catch (Exception e) { Debug.Log(e); } LogWriteBuffer(); WriteQueue queue = new WriteQueue(); queue.buffer = new byte[_BufferSize]; queue.buffer[0] = (byte)code; queue.buffer[1] = data1; queue.buffer[2] = data2; queue.buffer[3] = data3; queue.buffer[4] = 0xEE; queue.delay = delay; // _Queue.Add(queue); }</pre>
ReadBuffer / 종료 바이트 기반 패킷 처리	WriteBuffer / 장치 명령 전송
0xEE 종료 바이트 감지 읽기 버퍼 초과 시 Reset 예외 로그 중복 방지 ProcessReadBuffer 에서 명령 분기	SerialCode + data 3 바이트 + 종료 바이트 Timeout / IOException 분리 처리 카드 배출/장치 제어 명령 전송
수신 처리 기준	송신 처리 기준

데몬스쿼드 키우기 글로벌

사용 기술	Unity, C#, AOS/iOS, Firebase, Unity IAP, AdMob/IronSource, GPM	개발 정보	출시 프로젝트 유지보수 / 단독 담당
역할	기존 출시 프로젝트 퍼블리셔 슈퍼플래닛과 글로벌 대응을 단독 담당. 신규 개발보다는 유지보수, SDK 추가, 기능 추가, 최적화, 빌드 안정화 중심으로 기여.		
내용			

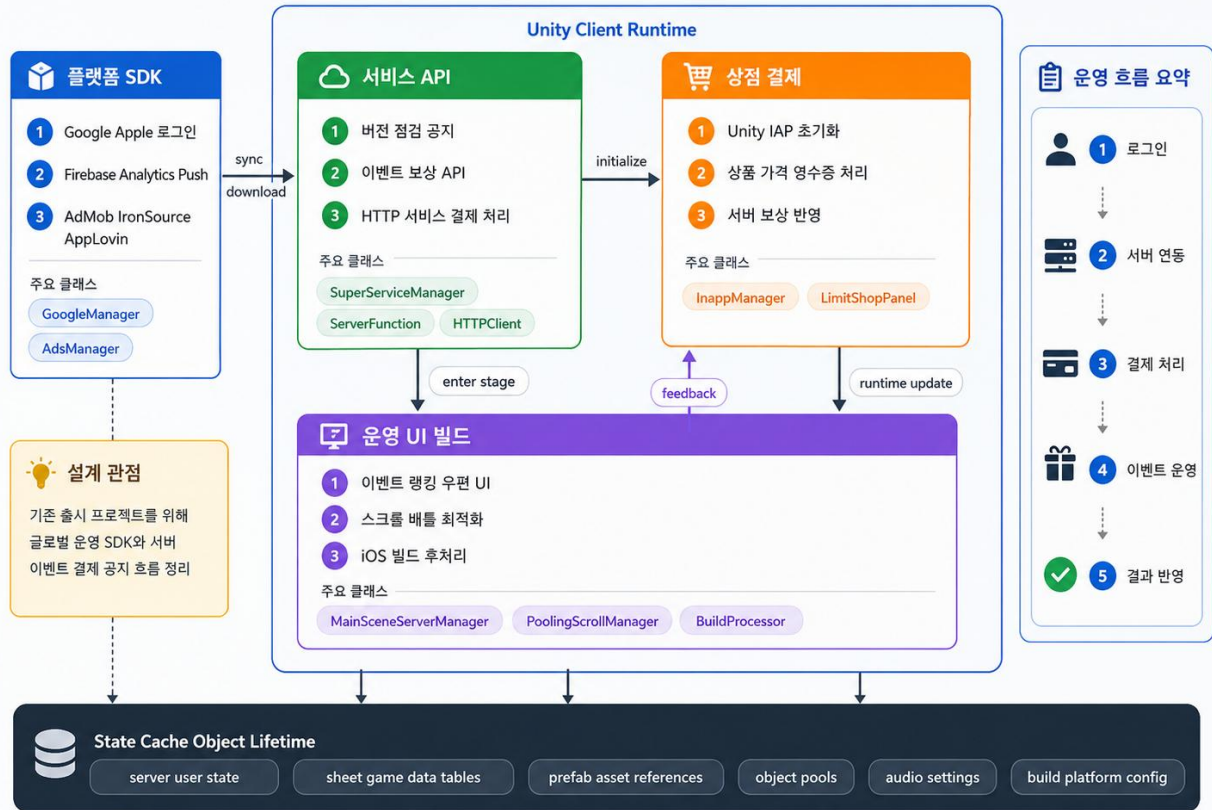
	프로젝트 소개 기존 출시 프로젝트의 글로벌 대응 작업입니다. SDK/광고/분석/푸시, IAP, 서버 이벤트, 운영 UI, AOS/iOS 빌드 대응과 최적화를 담당했습니다. 기존 게임코드를 읽어 출시 2 년이 지난게임을 출시 초기로 만들었습니다.
	관련 링크(스토어링크) https://play.google.com/store/apps/details?id=com.superplanet.demonsquad
	주요 작업 • Firebase, Crashlytics, Push, 광고 SDK, Unity IAP 연동 및 유지보수 • SuperService 기반 버전/공지/점검/로케일/보상 API 대응 • 이벤트, 랭킹, 상점, 패스, 월드레이드 등 운영 UI 기능 추가 • iOS 빌드 후처리와 플랫폼별 SDK 대응
	성과 • 기존 출시 프로젝트를 글로벌 서비스 환경에 맞게 유지보수하고 운영 안정성을 보완했습니다. • 서버, 결제, 광고, 분석, 푸시 흐름을 실제 서비스 기준으로 대응했습니다.

아키텍처

플랫폼 SDK, 서비스/API, 상점/결제, 운영 UI/빌드 계층으로 정리됩니다. Google/Firebase/광고 SDK는 플랫폼 계층에서 관리하고, SuperService와 ServerFunction은 운영 데이터와 서버 응답을 처리하며, 각 이벤트/상점 UI가 유저 상태를 갱신합니다.

데몬스쿼드 키우기 글로벌 운영 아키텍처

System architecture / live operations / runtime data flow



아키텍처 구조도

개발 내용

개발 내용	기능 구현 • 광고 보상, IAP 결제, 서버 보상 반영 흐름 유지보수 • 공지/점검/강제 업데이트/추천 업데이트 등 운영 기능 대응 • 이벤트, 랭킹, 상점, 월드레이드 등 글로벌 운영 UI 기능 추가
-------	--

최적화

- 반복 UI 패널과 스크롤 구조를 점검해 런타임 부담 완화
- SDK 초기화 순서와 서버 콜백 흐름을 정리해 오류 대응성 개선
- iOS 빌드 후처리를 자동화해 빌드 누락 가능성 감소

리팩토링

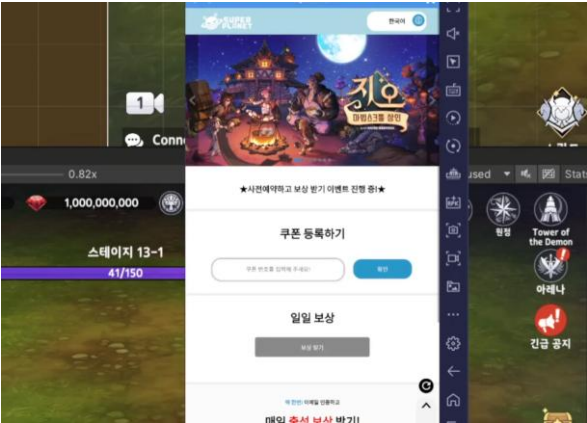
- 기존 구조를 유지하면서 신규 SDK와 운영 기능을 추가하는 방식으로 리스크를 낮춤
- 서버 응답 처리와 UI 반영 흐름을 분리해 기능 추가 범위를 관리

데몬스워드 키우기 글로벌 - 기능 이미지 / 코드 정리

기존 출시 프로젝트의 글로벌 대응 과정에서 GPM WebView, 광고/이벤트 연결, 로컬라이징 데이터 처리, 시즌패스 이벤트 기능을 유지보수했습니다.

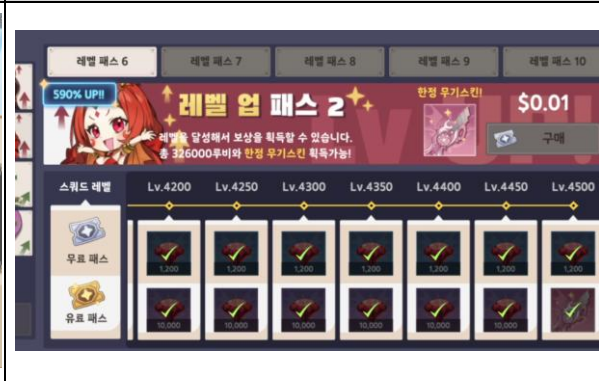
01. GPM WebView / Event

GPM WebView 로 외부 이벤트를 게임 안에서 열고, 닫기 콜백 이후 화면 방향을 복구하는 흐름입니다.

	
GPM WebView 이벤트 화면	GPM WebView 호출 / 콜백 처리 코드
Fullscreen WebView 표시 Close / Back / Forward 버튼 설정 CallbackType.Close 에서 방향 복구 Android/iOS 방향 처리 분기	글로벌 이벤트 페이지 연결 게임 화면 위 WebView 오픈 이벤트 종료 후 플레이 화면 복귀
WebView 처리 기준	이벤트 연결 흐름

02. Localization / Season Pass

글로벌 출시 대응을 위해 서버에서 언어 CSV 를 받아 저장하고, 선택 언어 컬럼을 기준으로 로컬라이징 딕셔너리를 구성했습니다.

<pre> public IEnumerator GetLocalization() { WWWForm form = new WWWForm(); form.AddField("client_key", ClientKey); form.AddField("locale", LocalizationString); using (UnityWebRequest request = UnityWebRequest.Post(SuperHost + "api/localization", form)) { request.SendWebRequest(); while (!request.isDone) { Debug.Log(\$"Download Progress: {request.downloadProgress * 100}%"); yield return null; } if (request.result == UnityWebRequest.Result.ConnectionError request.result == UnityWebRequest.Result.ProtocolError) { Debug.LogError("Error : " + request.error); StartCoroutine(ProcessError(request.responseCode)); } else { string directoryPath = Path.Combine(DirectoryName, savePath); if (!Directory.Exists(directoryPath)) { Directory.CreateDirectory(directoryPath); } if (File.Exists(savePath)) { File.Delete(savePath); Debug.Log("Deleted existing CSV file."); } // 새 파일 저장 File.WriteAllBytes(savePath, request.downloadHandler.data); yield return new WaitForSeconds(1) == File.Exists(savePath); Event("season_pass_init_and"); GetPlayerPrefs(); PlayerPrefs.Save(); StartCoroutine(LoadOurSceneAsync()); } } } </pre>	<pre> public IEnumerator LoadLocalizationFromServer() { var dict = new Dictionary<string, string>(); try { string[] lines = File.ReadAllLines(SuperServiceManager.Instance.savePath); string[] headers = ParseLine(lines[0]).ToArray(); int index = -1; for (int i = 0; i < headers.Length; i++) { if (headers[i].Contains(SuperServiceManager.Instance.LocalizationString)) { index = i; break; } } if (index == -1) { Debug.LogError("같은 언어가 없음"); return null; } // 각 행을 읽어와서 디코딩에 추가 for (int i = 1; i < lines.Length; i++) { List<string> data = ParseLine(lines[i]); if (data.Count > index) { string key = data[0].Replace(" ", ""); string value = data[index].Replace(" ", ""); if (value.Contains("₩")) value = value.Replace("₩", "\$"); if (DataManager.Instance.currency == CSV_Language_B0 value = value.Replace(" ", ""); dict[key] = value; } } GetLang = true; } catch (Exception e) { Debug.LogError("Error loading CSV file: " + e.Message); } StartCoroutine()?.Invoke(); return dict; } </pre>
언어 CSV 다운로드 / 저장 코드	선택 언어 컬럼 파싱 / 딕셔너리 구성
	
시즌패스 화면	레벨 패스 이벤트 화면

서브 프로젝트 상세 정리

메인 프로젝트보다 비중을 낮게 두고, 플랫폼 경험과 기능 구현 범위를 보여주는 보조 프로젝트만 간단히 정리했습니다.

VR / WebGL / Kiosk	Mobile / Firebase / AdMob	Editor Tool / Shader / Utility
--------------------	---------------------------	--------------------------------

	EGOISM 추리게임 기반 서비스형 클라이언트 기능 구현 PC / Mobile · 추리게임 · Photon · Firebase 기간/역할 2022.04 ~ 2023.03 / 클라이언트 기능 구현 기술 Unity, Photon, Firebase, REST API 프로젝트 설명 로그인, API 저장과 불러오기, Photon 룸과 채팅, CSV 로컬라이징을 담당한 4인 팀 추리게임 프로젝트입니다. 주요 작업 • Google Sheet CSV 기반 로컬라이징 적용 • Photon 룸, 채팅, 대기방 상태 표시 기능 구현 • 유저 데이터 저장, 불러오기, 설정값 관리 흐름 구현
--	---

대표 이미지



대표 이미지

메타버스 시흥

WebGL 메타버스 콘텐츠와 유저 동기화 구현

WebGL · React · Dead Reckoning · 국가사업

기간/역할 2022.04 ~ 2022.12 / 메인 개발자

기술 Unity WebGL, React, Addressables, Dead Reckoning

프로젝트 설명

갯골생태공원을 WebGL 메타버스로 구현한 프로젝트로, 아바타와 미니게임, 위치 동기화, 콘텐츠 로드 기능을 담당했습니다.

주요 작업

- 아바타 선택, 이동, 특정 구간 상호작용 기능 제작
- 게임 서버 없이 Dead Reckoning 방식으로 유저 위치 동기화
- Addressables 기반 맵 생성, 스크린샷, 관리자와 회의실 기능 제작

뉴스

<https://www.fnnews.com/news/202209201058025759>



대표 이미지

이모션티니핑

모바일 출시 대응과 Firebase, IAP 기능 대응

Android · iOS · Firebase · IAP · 뒤끝

기간/역할 2024.01 ~ 2024.06 / 서브 개발

기술 Firebase Auth, Messaging, Remote Config, Google SignIn, IAP, 뒤끝

프로젝트 설명

Android 와 iOS 모바일 서비스에서 Firebase 인증, 메시징, Remote Config, 인앱결제와 출시 대응 기능을 보조 개발했습니다.

주요 작업

- Google SignIn 과 Firebase Auth 연동 흐름 대응
- Firebase Messaging, Remote Config 등 서비스 SDK 적용
- 인앱결제와 iOS 출시
- 기능 보조 개발

링크

플레이스토어:

<https://play.google.com/store/apps/details?id=com.entereal.emotionteenieping>

앱스토어:

<https://apps.apple.com/kr/app/%EC%9D%B4%EB%AA%A8%EC%85%98%ED%8B%B0%EB%8B%88%ED%95%91/id6450882377>



대표 이미지

PhotoKiosk

촬영, 저장, 출력, 메일 전송 흐름을 구현한 키오스크

PC Kiosk · WebCam · Printer · Email

기간/역할 2023.01 ~ 2023.02 / 1 인 개발

기술 Unity, C#, WebCam, Printer, Email

프로젝트 설명

항일의병기념공원 Photo Zone 에서 사용하는 Unity 기반 키오스크를 1 인 개발로 구현했습니다.

주요 작업

- WebCam 촬영과 날짜 기반 이미지 저장 구현
- 저장 이미지 프린터 출력 후 임시 파일 정리
- 촬영 이미지를 이메일로 전송하는 흐름 구현

 대표 이미지	EPACSE VR 시야와 빛 상호작용을 활용한 공포 탈출 게임 PC VR · HTC Vive · G-STAR · 공포 기간/역할 2018.08 ~ 2018.11 / 기획 및 개발 기술 Unity3D, C#, HTC Vive 프로젝트 설명 귀신을 바라보거나 빛을 비추면 움직임이 멈추는 규칙을 설계한 PC VR 공포 게임입니다. 주요 작업 • VR 시야 판정 기반 Enemy 정지 규칙 구현 • 빛 상호작용으로 적 이동을 제한하는 플레이 규칙 구현 • G-STAR 2018 출품 및 STOVE VR 계약 경험
---	--

 대표 이미지	SCHOOL MAZE VR 컨트롤러 조작 기반 캐주얼 미로 게임 PC VR · HTC Vive · G-STAR · 미로 기간/역할 2019.09 ~ 2019.11 / 기획 및 개발 기술 Unity3D, C#, HTC Vive 프로젝트 설명 컨트롤러로 칠판 위 미로를 이동하며 벽과 추적 몬스터를 피하는 PC VR 캐주얼 게임입니다. 주요 작업 • VR 컨트롤러 기반 캐릭터 이동 구현 • 벽 충돌과 추적 AI 실패 조건 처리 • G-STAR 2019 출품 및 MonsterVR 제안 경험
---	---



대표 이미지

Parallel

모바일 공포 방탈출 플레이 기능 구현

Mobile · 공포 방탈출 · PlayerPrefs · 조이스틱

기간/역할 2021.03 ~ 2021.07 / 클라이언트 기능 구현

기술 Unity, Mobile UI, PlayerPrefs, Touch Input

프로젝트 설명

폐교를 배경으로 열쇠를 찾고 탈출하는 모바일 공포 방탈출 게임에서 플레이 기반 기능을 구현했습니다.

주요 작업

- 모바일 조이스틱과 카메라 회전 입력 구현
- 카메라 촬영과 귀신 정지, 배터리 소모 흐름 구현
- PlayerPrefs 기반 귀신 정보 확인 상태 저장과 스크린샷 기능 구현

링크

<https://play.google.com/store/apps/details?id=com.BJSH.Parallel>



대표 이미지

메모리편의점

퍼즐 미니게임과 광고 연동을 구현한 모바일 프로젝트

Mobile · 퍼즐게임 · AdMob · PlayerPrefs

기간/역할 2020.02 ~ 2020.05 / 미니게임 구현 및 AdMob

기술 Unity, Touch, PlayerPrefs, Google AdMob

프로젝트 설명

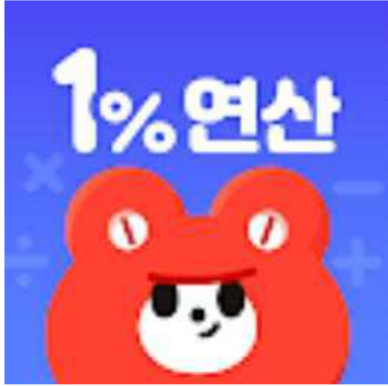
편의점 콘셉트의 퍼즐 모바일 게임에서 터치 조작, 스테이지 저장, 정답 로직, AdMob 연동을 담당했습니다.

주요 작업

- 터치 기반 오브젝트 이동과 배치 체크 구현
- PlayerPrefs 기반 스테이지 저장과 불러오기 구현
- 스테이지별 정답 버튼 세팅과 Google AdMob 광고 적용

링크

<https://play.google.com/store/apps/details?id=com.DC.PA>



대표 이미지

일프로연산 / 수학 교육앱

교육용 콘텐츠 개발과 반복 작업 자동화

Editor Tool · Shader · Utility · 유지보수

기간/역할 2020.12 ~ 2022.03 / 콘텐츠 개발 및 사수 역할

기술 Unity, C#, Shader, Editor Tool

프로젝트 설명

매쓰마스터 재직 중 교육용 앱 콘텐츠 개발과 프로젝트 공통 유틸, 에디터 자동화, UI 셰이더 작업을 수행했습니다.

주요 작업

- 파일 정리와 텍스처 이름 변경 자동화 Editor Tool 제작
- 제네릭 싱글톤과 애니메이션 유틸로 반복 코드 감소
- UI 흔들림, 물결, 페이드 계열 셰이더 제작

링크

<https://play.google.com/store/apps/details?id=com.Mathmaster.OneProMath&hl=ko>